

Semana 17 - Rede Transformer

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

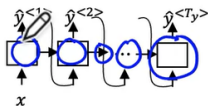
- 1 Intuição
- 2 Auto-atenção
- 3 Atenção Multi-Cabeças
- 4 Rede Transformer

Intuição da Rede Transformer

- Um dos desenvolvimentos mais interessantes do *deep learning*
- Muitos dos algoritmos mais efetivos de NLP atuais são baseados nos Transformers

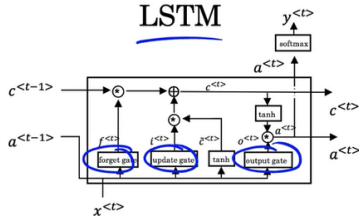
increased complexity
sequential

RNN



GRU

LSTM



Intuição da Rede Transformer

- À medida que a complexidade da tarefa de sequência aumenta, a complexidade do modelo também aumenta
- Embora modelos mais avançados como GRUs e LSTMs tenham melhorado o controle sobre o fluxo de informação, eles também adicionaram complexidade
- Todos esses modelos ainda são sequenciais, uma vez que eles consomem a entrada, por exemplo em uma sentença, uma palavra de cada vez
- Cada bloco na figura se torna um gargalo, já que para calcular a saída todos os blocos que vêm antes precisam ser calculados
- Na arquitetura Transformer esse processamento é paralelizado, a sentença inteira é lida de uma vez

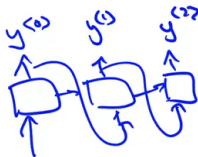
Transformer¹

- AUTORES: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin
- CURIOSIDADE: Um dos autores, o Lucasz Kaiser, é co-instrutor da especialização em NLP da deeplearning.ai.

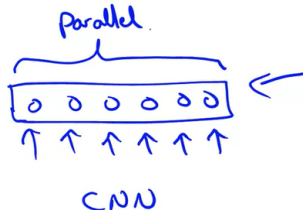
• Attention + CNN

- Self-Attention

- Multi-Head Attention



$A^{(1)}$ $A^{(2)}$ $A^{(3)}$ $A^{(4)}$ $A^{(5)}$



¹Vaswani et al. 2017, Attention is All You Need.

Transformer

- A principal inovação do Trasnformer é combinar a representação baseada em atenção com o estilo de processamento da CNN
- Trocamos o processamento sequencial pelo paralelo
- Duas ideias serão fundamentais na rede Transformer
- A primeira é a **auto-atenção**
- Imagine uma sentença de 5 palavras; A auto-atenção vai calcular uma representação usando atenção para cada palavra, em paralelo
- Temos também a **atenção multi-cabeças**, que gera múltiplas versões dessas representações

Outline

- 1 Intuição
- 2 Auto-atenção**
- 3 Atenção Multi-Cabeças
- 4 Rede Transformer

Auto-atenção - Intuição

$A(q, K, V)$ = attention-based vector representation of a word

↳ calculate for each word $A^{(1)}, \dots, A^{(s)}$

RNN Attention

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^T x \exp(e^{<t,t'>})}$$

$A^{(3)}$

$x^{<1>}$ Jane
 $x^{<2>}$ visite
 $x^{<3>}$ l'Afrique

Transformers Attention

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

$x^{<4>}$ en
 $x^{<5>}$ septembre
 $q^{<3>}$ $k^{<3>}$ $v^{<3>}$

Auto-atenção - Intuição

- A ideia principal por trás do Transformer é o mecanismo de auto-atenção
- Já vimos o uso de atenção em modelos de sequência
- Para modelos no estilo da CNN precisamos do conceito de auto-atenção, em que uma representação baseada em atenção é criada para cada palavra da sentença de entrada
- Vamos voltar à nossa sentença de exemplo e criar as representações baseadas em atenção $A^{<1>}, \dots, A^{<5>}$, já que nossa sentença tem 5 palavras

Auto-atenção - Intuição

- Agora para representar uma palavra como “l’Afrique” vamos levar em conta também o contexto na palavra na sentença, se estamos falando da África como um lugar de interesse histórico, turístico, o maior continente do mundo, etc.
- O que nossa representação faz é olhar para as palavras em volta na tentativa de capturar esse contexto
- Em termos de cálculos, nem é tão diferente do mecanismo de atenção que já vimos na última semana
- A diferença é que agora vamos calcular uma representação em paralelo para as 5 palavras

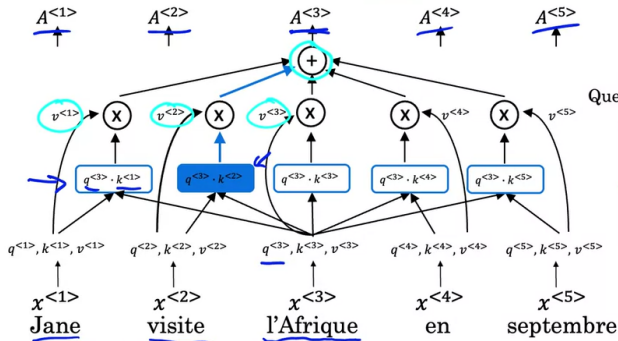
Auto-atenção - Intuição

- Isso faz também com que a equação de atenção mude, e não usamos mais a fórmula de energias da RNN
- Porém elas ainda possuem semelhanças; O termo interno também envolve softmax e os termos com exponenciais podem ser pensados como valores de atenção
- Mas a principal diferença mesmo é que agora para cada palavra (ex. "l'Afrique") temos 3 vetores: *query*, *key* e *value*
- Esses vetores são elementos-chave no cálculo dos valores de atenção de cada palavra

Auto-atenção - Cálculo

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Query (Q)	Key (K)	Value (V)
$q^{<1>}$	$k^{<1>}$ person	$v^{<1>}$
$q^{<2>}$	$k^{<2>}$ action	$v^{<2>}$
$q^{<3>}$ what's happening there	$k^{<3>}$	$v^{<3>}$
$q^{<4>}$	$k^{<4>}$	$v^{<4>}$
$q^{<5>}$	$k^{<5>}$	$v^{<5>}$

$$\begin{aligned} q^{<3>} &= W^Q \cdot x^{<3>} \\ k^{<2>} &= W^K \cdot x^{<2>} \\ v^{<2>} &= W^V \cdot x^{<2>} \end{aligned}$$

Auto-atenção - Cálculo

- Vamos ilustrar com o cálculo de $A^{<3>}$ (“l’Afrique”)
- Cada palavra vai ser associada a 3 vetores *query*, *key* e *value*
- Cada um desses vetores são calculados a partir de matrizes W^Q , W^K e W^V , que tem seus valores como parâmetros aprendidos
- No exemplo da 3ª palavra, cujo *embedding* é $x^{<3>}$, as fórmulas são:

$$q^{<3>} = W^Q x^{<3>}$$

$$k^{<3>} = W^K x^{<3>}$$

$$v^{<3>} = W^V x^{<3>}$$
- Essas definições possuem uma vaga conexão com bancos de dados, em que há uma *query* (consulta) e pares de *key-value* para a resposta
- Podemos pensar também em $q^{<3>}$ como uma questão que podemos fazer sobre a África, p.ex., o que está acontecendo lá?

Auto-atenção - Cálculo

- Isso é interessante já que a África é um destino turístico e portanto quando estiver calculando $A^{<3>}$ seria interessante saber o que ocorre lá no momento
- E vamos calcular então o produto interno entre $q^{<3>}$ e cada uma das chaves $k^{<1>}$, $k^{<2>}$, $k^{<3>}$, etc.
- Essas chaves cumprem o papel de respostas para a questão $q^{<3>}$
- Em seguida calculamos o produto interno entre $q^{<3>}$ e $a^{<2>}$ isso nos diz o quanto a palavra “visite” é uma boa resposta para o que está acontecendo na África
- E assim por diante para as demais palavras

Auto-atenção - Cálculo

- O objetivo é extrair o máximo necessário de informação para gerar uma representação $A^{<3>}$ que seja o mais útil possível
- Apenas como uma intuição, vamos supor que $k^{<1>}$ representa que a 1ª palavra é uma pessoa (porque “Jane” é uma pessoa)
- E que $k^{<2>}$ corresponda ao fato de a segunda palavra ser uma ação (como “visite”)
- Então é de se esperar que o produto interno de $q^{<3>}$ com $k^{<2>}$ tenha o maior valor
- Isso sugere intuitivamente que “visite” provê o contexto mais relevante para o que está ocorrendo na África, i.e., ela é vista como um destino de visita

Auto-atenção - Cálculo

- Os 5 produtos internos calculados no caso são submetidos então a uma operação de softmax
- Na figura, o produto interno de $q^{<3>}$ com $k^{<2>}$ está realçado em azul por ter o maior valor
- E então multiplicamos o resultado da softmax por $v^{<1>}$ que é o *value* para a palavra 1, $v^{<2>}$, etc.
- Por fim, somamos tudo e obtemos $A^{<3>}$
- Note que $A^{<3>}$ pode ser escrito também como $A(q^{<3>}, K, V)$
- Uma grande vantagem dessa representação é que ela permite que as palavras não fiquem limitadas ao significado de word embedding fixo

Auto-atenção - Cálculo

- Pode-se associar por exemplo que “l'Afrique” é um destino de “visite”; E isso se aplica, claro, a todas as 5 palavras da sentença de exemplo
- A fórmula também possui uma versão vetorizada, em termos das matrizes Q , K e V :

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

- O denominador no caso é apenas para garantir que o produto interno não exploda
- Um outro nome para essa definição é “scaled dot-product attention” e é o que se usa no artigo original do Transformer, “Attention is all you need”

Auto-atenção - Cálculo

- Em resumo, temos agora associado a cada palavra uma “*query*”, uma “*key*” e um “*value*”
- A *query* faz o papel de uma pergunta (“O que está acontecendo na África?”)
- A *key* fornece, usando similaridade, a palavra que tem maior chance de dar uma resposta relevante para a *query* (no caso “visite”)
- Por fim, o *value* nos diz como “visite” deve ser representado dentro da representação $A^{<3>}$ para “l’Afrique”

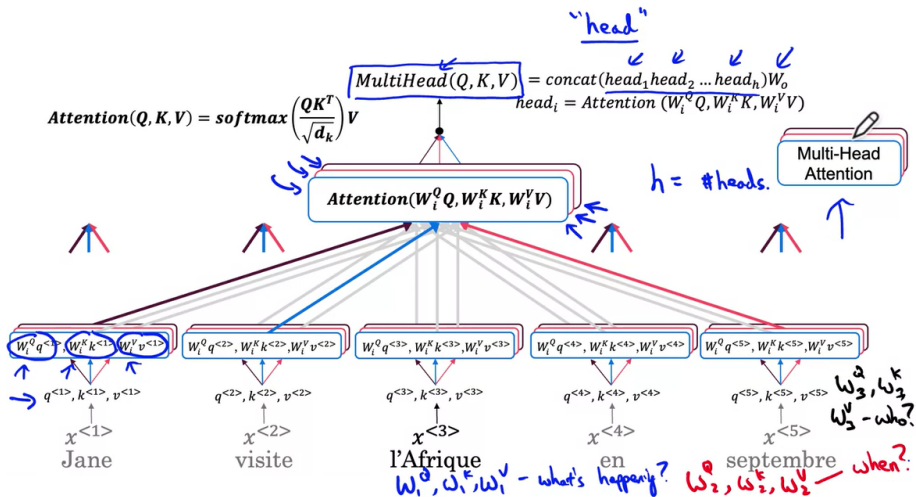
Auto-atenção - Cálculo

- É como se no final a representação $A^{<3>}$ dissesse “isto é a África e alguém está visitando a África”
- Ou seja, temos agora um contexto e uma representação muito mais flexível do que um *embedding* fixo

Outline

- 1 Intuição
- 2 Auto-atenção
- 3 Atenção Multi-Cabeças**
- 4 Rede Transformer

Atenção Multi-Cabeças



Atenção Multi-Cabeças

- Basicamente um laço *for* sobre o mecanismo de atenção
- Cada vez que calculamos a atenção como vimos temos uma “head”
- Na 1ª vez que rodamos (1ª cabeça) aplicamos as mesmas fórmulas já vistas, porém com as matrizes W_1^Q , W_1^K e W_1^V para as *queries*, *keys* e *values*
- Intuitivamente, podemos pensar, como no exemplo anterior, que essas matrizes estão relacionadas ao aprendizado da pergunta “O que está acontecendo?” e respectiva resposta e assim na representação de “l'Afrique” o maior produto interno seria para “visite” (seta azul na figura)
- A diferença é que vamos agora fazer isso várias vezes (8 cabeças por exemplo)

Atenção Multi-Cabeças

- Na segunda cabeça usamos subscrito 2, e as matrizes são portanto W_2^Q , W_2^K e W_2^V
- Essa segunda cabeça formularia uma segunda pergunta, por exemplo, “Quando (algo acontece)?”
- E neste caso, a palavra “septembre” poderia ter o maior produto interno na representação de “l’Afrique” (seta vermelha)
- A terceira pergunta, representada por W_3^Q , W_3^K e W_3^V poderia ser “Quem (tem algo a fazer na África)?”
- Neste caso, é provável que “Jane” tenha o maior produto interno entre sua *key* ($k^{<1>}$ com a *query* de “l’Afrique” $q^{<3>}$) e esta então teria maior peso na representação $A^{<3>}$ calculada na 3ª cabeça

Atenção Multi-Cabeças

- Na literatura é comum que se represente o número de cabeças por h
- E cada cabeça pode ser pensada como diferentes *features* que podem ser passados para uma nova rede para calcular uma representação muito rica da sentença
- O valor final da atenção multi-cabeças corresponde à concatenação dos valores em cada cabeça seguida pela multiplicação por uma matriz W_0
- NOTA: Embora tenhamos falado e pensado em um laço *for*, na implementação de fato, todos esses cálculos de cabeças são paralelizados, já que são independentes

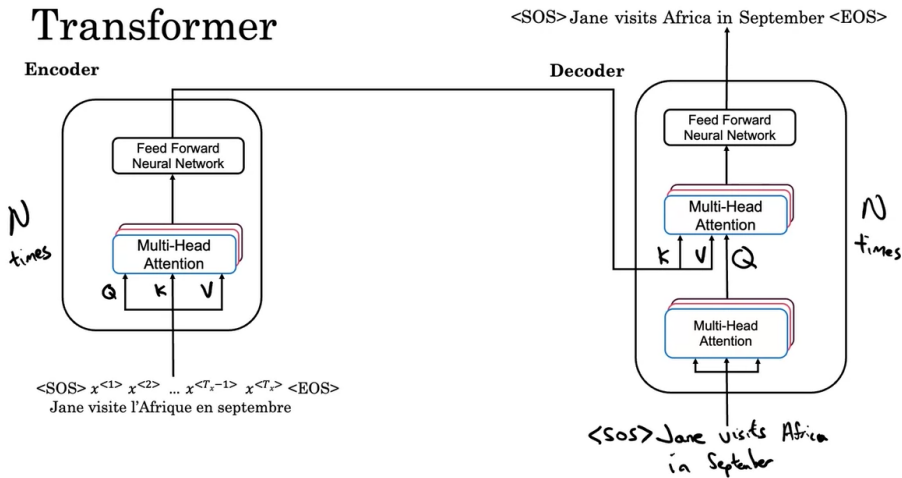
Atenção Multi-Cabeças

- NOTA 2: No cálculo de q , k e v multiplicávamos x por matrizes W
- Isso não é mais necessário no caso multi-cabeças porque já temos as matrizes W_i de cada cabeça e então estaríamos multiplicando duas vezes
- Na verdade, na versão mais simples da atenção multi-cabeças temos $q = k = v = x$
- Aqui mantemos q , k e v como valores diferentes apenas porque em uma parte do Transformer (quando se calcula a atenção entre a entrada e saída), eles carregam informação diferente
- NOTA 3: O processo todo da atenção multi-cabeças é muito denso e por isso vamos adotar um ícone simplificado de pilha para representá-lo

Outline

- 1 Intuição
- 2 Auto-atenção
- 3 Atenção Multi-Cabeças
- 4 Rede Transformer**

Transformer



Transformer

- Auto-atenção + atenção multi-cabeças = Rede Transformer
- Temos um *token* para *start of sentence* ($\langle SOS \rangle$) e *end of sentence* ($\langle EOS \rangle$)
- O 1º passo é ler a sentença em um *Encoder*, que contém um módulo de atenção multi-cabeças
- Esse módulo gera uma matriz que é usada como entrada para uma rede *feedforward*, que captura *features* importantes da sentença
- Na Rede Transformer esse bloco de *encoder* é repetido N vezes ($N = 6$ é um valor típico)

Transformer

- A saída do *encoder* é usada então como entrada do bloco *decoder*
- Este deve gerar a sentença traduzida e a primeira palavra deve ser $\langle \text{SOS} \rangle$
- A parte da sentença traduzida já gerada é usada como entrada em um bloco multi-cabeças no *decoder*
- No início da tradução, temos apenas o *token* $\langle \text{SOS} \rangle$ e é esse que é usado então para calcular Q , K e V
- A saída desse bloco multi-cabeças dá origem à matriz Q de um próximo bloco multi-cabeças no *decoder*

Transformer

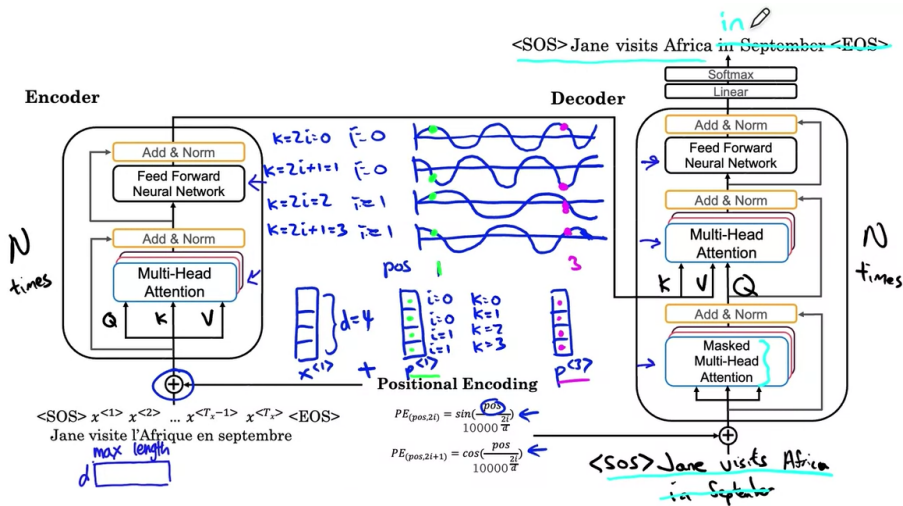
- As demais matrizes K e V , por sua vez, são recebidas do *encoder*
- A intuição é que temos uma *query* (Q) que é basicamente “O que eu já tenho da sentença traduzida?”
- Ao mesmo tempo eu tenho um contexto (K e V) que vem da sentença em francês; E isso ajuda a decidir qual a próxima palavra na sequência
- Por fim, esse segundo bloco multi-cabeças no *decoder* é usado como entrada em mais uma rede *feedforward*
- O bloco do *decoder* também é repetido N vezes (e.g., $N = 6$), sendo que a saída de cada execução é retro-alimentada como entrada na próxima execução

Transformer

- Essa rede feedforward é responsável por decidir a 1ª palavra na sentença em inglês (que esperamos seja “Jane”)
- E então “Jane” é acrescentada ao $\langle \text{SOS} \rangle$ como uma nova entrada ao *decoder*
- E então a pergunta que este módulo tenta responder é: dado “Jane”, qual a próxima palavra mais adequada? Quais valores de K e V para essa melhor escolha (que esperamos ser “visits”)?

E assim por diante as demais palavras vão sendo geradas e acrescentadas também à entrada do *decoder* até que, esperamos, se obtenha o *token* $\langle \text{EOS} \rangle$

Transformer - Detalhes



Transformer - Detalhes

- Mas temos ainda alguns detalhes no modelo Transformer que são bastante importantes
- O primeiro é o *positional encoding* da entrada
- Note que as equações de auto-atenção não usam nenhuma informação da posição da palavra na frase
- Mas esse dado pode ser fundamental em um processo de tradução
- Uma fórmula típica para isso usa seno e cosseno
- Imagine que seu *word embedding* tenha dimensão 4
- Criamos então um vetor *positional encoding* também de dimensão 4

Transformer - Detalhes

- Na fórmula, pos é a posição da palavra na sentença, e.g., “Jane” tem $pos = 1$
- Já i se refere a posição no *positional encoding*, $i = 0, 0, 1, 1$, em que os valores se repetem 2 vezes porque os mesmos são usados no seno e no cosseno
- d é a dimensão do *positional encoding*, no caso, 4
- E assim temos vetores p diferentes para cada palavra, e.g., $p^{<1>}$ (“Jane”) e $p^{<3>}$ (“l’Afrique”) vão ser diferentes
- Veja na figura as curvas de senos e cossenos usadas
- Quando elas têm a mesma frequência ($k = 0, 1$), elas estão completamente fora de fase

Transformer - Detalhes

- Para $i = 1$, temos uma senoide de frequência mais baixa e depois uma cossenoide, ortogonal e com a mesma frequência
- Fato é que se eu ler os valores nessas (co)senoides eu terei sempre vetores diferentes para cada posição (palavra)
- O *positional encoding* $p^{<1>}$ é somado diretamente com $x^{<1>}$ e o mesmo para as demais palavras, de modo que o vetor da palavra seja também influenciado por sua posição na frase
- A dimensão da saída da camada de *embedding* é d (no nosso caso 4) pelo máximo comprimento de sequência suportado pelo modelo
- A saída dos módulos no *encoder* e *decoder* também terão esse formato

Transformer - Detalhes

- Além do *embedding*, o *positional encoding* também é passado para os módulos internos da rede por conexões residuais (similares à ResNet)
- O objetivo é passar uma informação posicional longa para a arquitetura inteira
- A Rede Transformer também emprega uma camada chamada de *Add & Norm* em vários pontos da arquitetura, que é muito similar à *BatchNorm*, que já vimos e ajuda a acelerar o treinamento
- Por fim, na saída do *encoder* temos uma camada linear e uma softmax, que prediz a próxima palavra da tradução, uma de cada vez
- Outro conceito que aparece na literatura do Transformer é o de
- Isso é importante apenas no treino

Transformer - Detalhes

- Lembre-se que no treinamento temos acesso à sentença correta inteira, tanto em francês quanto inglês
- Por ter a sentença inteira, não há necessidade, no treinamento, de se gerar uma palavra por vez
- Em vez disso, o *Masked Multi-Head* esconde a parte final da sentença, e.g., “in Septembre” e tenta forçar a rede a prever a tradução correta mesmo com essa parte escondida
- Desde o artigo original do Transformer tivemos também outras iterações, como BERT e BERT-Distill